

Class XII | Unit 3 | weight 25%

Object-oriented programming in C++

Board weight about 25% | 30 theory periods and 30 lab periods

This unit carries a quarter of Class XII. A class describes a kind of object. An object is one real instance. Encapsulation, inheritance and polymorphism are the three ideas the paper expects you to define, to distinguish, and to show in code.

What you should be able to do

- Explain encapsulation, inheritance and polymorphism, and the relationship of class and object.
- Use private and public members, and protected when a derived class must see them.
- Write constructors, including overloading, and a destructor.
- Write a derived class that uses members of its base class.
- Overload member functions and override a base function in a derived class.

Path through the lecture

- 1 Build the Student class on the first day: private data, public functions, one object.
- 2 Add a default constructor and a parameterised constructor with the Time class.
- 3 Inherit DriveClass from Base and show protected data being used.
- 4 Finish with one class that overloads area, and a derived class that overrides show.

3.1 Classes, objects and the three principles

Object-oriented programming. A way of organising a program as objects. An object combines data and the functions that work on that data. Classes describe those objects.

Class. A user-defined type that specifies the data members and member functions common to a kind of object.

Object. One instance of a class. It has its own copies of the data members and it can call the member functions.

Encapsulation places data and the functions that use it inside a class, and controls who may touch the data. **Inheritance** lets a new class receive members from an existing class, so common code is written once. **Polymorphism** lets one name mean more than one function, chosen from the arguments or from the kind of object.

A class has **data members**, the values stored in each object, and **member functions**, the operations. Data members are usually private. Member functions that the rest of the program may call are public. The public functions are the interface. The private data is the detail that can change without rewriting every caller.

3.2 Encapsulation and access

Access specifiers

Specifier	Who can use the member
private	Member functions of the same class. This is the default if you write nothing.
public	Any code that has the object.
protected	The class itself and classes derived from it. Not the general caller.

Data hiding. Keeping data members private so that outside code cannot change them directly. Outside code calls a public function, and that function can check the value.

A **constructor** is a member function with the same name as the class and no return type. It runs automatically when an object is created, so the object starts in a known state. A **default constructor** takes no arguments. A **parameterised constructor** takes values and copies them into the data members. **Constructor overloading** means providing more than one constructor, with different parameter lists. A **destructor** has the class name with a tilde in front, such as `~Time()`, takes no arguments, and runs when the object is destroyed. Use it to release anything the object acquired. In the simple classes in this unit the destructor is often empty, but you should be able to write one.

Program 3.1 Class Student with private data and public functions

```
#include <iostream>
using namespace std;

class Student {
private:
    int age;
    float percentage;
public:
    void setData(int a, float p) {
        age = a;
        percentage = p;
    }
    void showData() {
        cout << "Age: " << age << endl;
        cout << "Percentage: " << percentage << endl;
    }
};

int main() {
    Student s;
    int age;
    float percentage;
    cout << "Enter age and percentage: ";
    cin >> age >> percentage;
    s.setData(age, percentage);
    s.showData();
    return 0;
}
```

Main cannot write `s.age = 17` if age is private. It calls `setData`, and `showData` prints the values. That is encapsulation. Input 17 and 88 prints those values through the object, not through the variables in main.

Program 3.2 Default and parameterised constructors

```
#include <iostream>
using namespace std;

class Student {
private:
    int age;
    float percentage;
public:
    Student() {
        age = 0;
        percentage = 0;
    }
    Student(int a, float p) {
        age = a;
        percentage = p;
    }
    void showData() {
        cout << "Age: " << age << endl;
        cout << "Percentage: " << percentage << endl;
    }
};

int main() {
    Student s(17, 91);
    s.showData();
    return 0;
}
```

Program 3.3 Class Time with two constructors

```
#include <iostream>
using namespace std;

class Time {
private:
    int hours;
    int minutes;
    int seconds;
public:
    Time() {
        hours = 0;
        minutes = 0;
        seconds = 0;
    }
    Time(int h, int m, int s) {
        hours = h;
        minutes = m;
        seconds = s;
    }
    void show() {
        cout << "Time = " << hours << ":" << minutes
            << ":" << seconds << endl;
    }
};

int main() {
    int h, m, s;
    cout << "Enter hours, minutes and seconds: ";
    cin >> h >> m >> s;
    Time t(h, m, s);
    t.show();
    Time empty;
    empty.show();
    return 0;
}
```

Time t(h, m, s) calls the constructor with three arguments. Time empty; calls the default constructor and prints 0:0:0. The two constructors are an overload. There is no return type in front of Time.

3.3 Inheritance

Inheritance. A mechanism by which a derived class receives members from a base class and can add its own members.

Reusability is the practical gain. The base class holds the common members. Each derived class adds what is special to it, instead of copying the common code. The **base class** is the existing class. The **derived class** is the new class, written class DriveClass : public Base. Public inheritance means the public members of the base stay public for users of the derived class.

Protected members are the ones a derived class may use while the rest of the program still cannot. In the program below, name and age are protected in Base, so DriveClass can rely on the base functions that set them. Roll belongs only to the derived class.

Program 3.4 A derived class using a base class

```
#include <iostream>
#include <string>
using namespace std;

class Base {
protected:
    string name;
    int age;
public:
    void setBase(string n, int a) {
        name = n;
        age = a;
    }
    void showBase() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

class DriveClass : public Base {
private:
    int roll;
public:
    void setDrive(string n, int a, int r) {
        setBase(n, a);
        roll = r;
    }
    void showDrive() {
        showBase();
        cout << "Roll: " << roll << endl;
    }
};

int main() {
    DriveClass student;
    string name;
    int age, roll;
    cout << "Enter name, age and roll: ";
    cin >> name >> age >> roll;
    student.setDrive(name, age, roll);
    student.showDrive();
    return 0;
}
```

The object in main is a DriveClass, not a Base. setDrive calls setBase, so the derived class reuses the base behaviour. The printed name, age and roll show both parts of the object.

3.4 Polymorphism

Polymorphism. One interface, more than one function. In this course that means overloading and overriding.

Overloading a member function means several functions in the same class share a name and differ in their parameter lists. `area(int)` and `area(int, int)` can both exist. The call decides which one runs. This is the same rule as function overloading in Class XI.

Overriding means a derived class provides its own version of a function that the base class already has. The derived version replaces the base version for a derived object. If the base function is marked `virtual` and the call is made through a base pointer that actually points to a derived object, the derived function runs. That is the form of overriding that behaves according to the real object. A function that is not virtual and is called through a base pointer would run the base version, so the examples in this lecture mark show as virtual.

Program 3.5 Overloading area and overriding show

```
#include <iostream>
using namespace std;

class Base {
public:
    void area(int side) {
        cout << "Square = " << side * side << endl;
    }
    void area(int length, int width) {
        cout << "Rectangle = " << length * width << endl;
    }
    virtual void show() {
        cout << "Base show" << endl;
    }
};

class Drive : public Base {
public:
    void show() override {
        cout << "Drive show" << endl;
    }
};

int main() {
    Base shape;
    shape.area(4);
    shape.area(4, 5);
    Drive d;
    Base *p = &d;
    p->show();
    return 0;
}
```

`shape.area(4)` prints 16. `shape.area(4, 5)` prints 20. The pointer `p` has type `Base*` and points at a `Drive` object. Because `show` is virtual, the call prints `Drive show`. If you remove `virtual`, the same call prints `Base show`. That single experiment is the best way to remember why the keyword is there.

Overloading and overriding

Basis	Overloading	Overriding
Where	Same class	Derived class and its base
Names	Same name	Same name
Parameter list	Must differ	Stays the same
What chooses the function	The arguments at the call	The kind of object, when the function is virtual

Exam point

A full-mark class program has: the class heading, private data, a public constructor or setter, a public display function, an object in main, a call, and matching braces. Inheritance adds the colon, public Base, and at least one use of a base member.

Key terms

Term	Meaning to write in an answer
Class	The description of data members and member functions.
Object	One instance of a class.
Encapsulation	Data and functions bound together, with access controlled.
Data hiding	Private data changed only through member functions.
Constructor	A special function that initialises an object when it is created.
Destructor	A special function that runs when an object is destroyed.
Base class	The class that is inherited from.
Derived class	The class that inherits.
Overriding	A derived class replacing a base function of the same signature.

Bring it together

- A class is a type. An object is one value of that type.
- Private data and public functions are encapsulation. Protected data is visible to derived classes.
- Constructors initialise. You may overload them. A destructor tidies up.
- Inheritance reuses a base class. Polymorphism in this unit is overloading plus overriding.

Do this in the practical

- 1 Define Student with age and percentage. Input the values and show them through an object.
- 2 Initialise a Student and a Time object with constructors.
- 3 Inherit a derived class from a base class and show data through the derived object.
- 4 Demonstrate overloading of area and overriding of show.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. An object is:

- A. An instance of a class
- B. A transmission medium
- C. A file mode
- D. A normal form

2. Private members can be used directly by:

- A. Member functions of the same class
- B. Every function in the program
- C. Only the operating system
- D. Only SQL

3. A constructor:

- A. Has the class name and no return type
- B. Is named main
- C. Must return int
- D. Is a topology

4. Public inheritance is written:

- A. class Drive : public Base
- B. class Drive inherits Base
- C. Drive extends public
- D. public(Drive, Base)

5. Protected members are available to:

- A. The class and its derived classes
- B. Every file on the disk
- C. Only the linker, never a derived class
- D. Only constructors of unrelated classes

6. Overloading is distinguished by:

- A. The parameter list
- B. The return type alone
- C. The file name
- D. The colour of the IDE

7. A virtual function lets a base pointer call:

- A. The derived override when the object is derived
- B. Only ROM
- C. A flowchart
- D. The power supply

8. The destructor name for class Time is:

- A. ~Time
- B. delete Time
- C. Time()
- D. end Time

Answer key

Q	Answer	Why this is the answer
1	A	The class describes the kind. The object is one instance.
2	A	Private means the rest of the program must go through the class's own functions.
3	A	Constructors are named after the class and do not have a return type.
4	A	The colon and public Base are the C++ syntax.
5	A	Protected sits between private and public for inheritance.
6	A	Overloaded functions share a name and differ in their parameters.
7	A	virtual selects the function from the actual object.

Q	Answer	Why this is the answer
8	A	The destructor is a tilde followed by the class name.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Differentiate a class and an object.

Model answer

A class is a user-defined type that describes data members and member functions. An object is one instance of that class and holds its own data. Student is the class. One pupil, s, is an object. Many objects can be created from one class.

Q2. Define encapsulation and data hiding.

Model answer

Encapsulation binds data and the functions that operate on it into a class, and uses access specifiers to control access. Data hiding keeps the data private so outside code cannot assign to it directly. A public function can check a value before storing it.

Q3. Differentiate a constructor and a destructor.

Model answer

A constructor runs when an object is created and initialises it. It has the class name and no return type. A destructor runs when the object is destroyed and releases what the object acquired. Its name is ~ followed by the class name. A class may overload constructors. There is one destructor.

Q4. What does public inheritance give the derived class?

Model answer

The derived class receives the members of the base and can add its own. Public members of the base remain available to users of the derived object. Protected members of the base can be used inside the derived class. Private members of the base stay accessible only through the base class's own functions.

Q5. Differentiate overloading and overriding.

Model answer

Overloading defines several functions of the same name in one class with different parameter lists. The arguments select the function. Overriding defines, in a derived class, a function with the same signature as a base function. For a derived object the derived function replaces the base one. Mark the base function virtual when the call is made through a base pointer.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain encapsulation, inheritance and polymorphism, and illustrate each with C++.

- 1 Define OOP, class and object.
- 2 Explain encapsulation with private data and a public set and show function. Mention the access specifiers.
- 3 Explain inheritance with base, derived, public inheritance and reusability. Mention protected.
- 4 Explain polymorphism as overloading and overriding, with the virtual keyword for a real override through a pointer.
- 5 Use one small sketch of code for each idea rather than one unrelated program.

Question 2. Explain constructors, constructor overloading and the creation of objects. Use the class Time.

- 1 Define a constructor and state that it has no return type.
- 2 Write a default constructor that stores zeroes and a constructor that copies hours, minutes and seconds.
- 3 Show Time empty; and Time t(h, m, s); and say which constructor each one calls.
- 4 Show a display function.
- 5 Add one sentence on the destructor and when it runs.